

Лабораторная работа №3

Решение систем линейных уравнений методом треугольной факторизации

Выполнила Костылева Э.П.

Постановка задачи:

Разработать программу для решения систем линейных уравнений методом треугольной факторизации.

Математическая модель:

1. Нахождение корней СЛУ последовательным решением 2-х систем с полученными треугольными матрицами L и R

Handwritten mathematical formulas on grid paper:

$$LZ = B \quad \text{и} \quad RX = Z$$
$$z_1 = \frac{b_1}{l_{11}}; \quad z_i = \left(b_i - \sum_{k=1}^{i-1} l_{ik} z_k \right) / l_{ii}$$

$i = 2 \div n$

$$x_n = z_n; \quad x_i = z_i - \sum_{j=i+1}^n r_{ij} x_j$$

$i = (n-1) \div 1$

2. Перемножение матриц

$$\gamma_{ii} = 1$$

$$l_{ti} = a_{ti} - \sum_{j=1}^{i-1} l_{tj} \gamma_{ji} \quad i =$$

$$l_{ti} = a_{ti} - \sum_{j=1}^{i-1} l_{tj} \gamma_{ji}$$

$$\gamma_{it} = \left(a_{it} - \sum_{j=1}^{i-1} l_{ij} \gamma_{jt} \right) / l_{ti}$$

Код программы:

```

1  #Метод треугольной факторизации
2
3  import numpy as np
4  from tabulate import tabulate
5
6  #Ввод данных
7  n = 4
8
9  AB = np.array([[5, 7, 6, 5, 23], [7, 10, 8, 7, 32], [6, 8,
10 10, 9, 33],
11  |         |         |         |         |
12  |         |         |         |         |
13  |         |         |         |         |
14  |         |         |         |         |
15  |         |         |         |         |
16  |         |         |         |         |
17  |         |         |         |         |
18  |         |         |         |         |
19  |         |         |         |         |
20  |         |         |         |         |
21  |         |         |         |         |
22  |         |         |         |         |
23  |         |         |         |         |
24  |         |         |         |         |
25  |         |         |         |         |
26  |         |         |         |         |
27  |         |         |         |         |
28  |         |         |         |         |
29  |         |         |         |         |
30  |         |         |         |         |
31  |         |         |         |         |
32  |         |         |         |         |
    [5, 7, 9, 10, 31]])
11
12  print("Матрица A: ")
13  print(tabulate(AB, tablefmt="fancy_grid"))
14
15  #Матрица A
16  A = np.array([[5, 7, 6, 5], [7, 10, 8, 7], [6, 8, 10, 9], [5,
17 7, 9, 10]])
18
19  B = np.array([23, 32, 33, 31])
20
21  def matrix_l(i, j):
22      if j > i:
23          return 0
24      res = A[i][j]
25      if j == 0:
26          return res
27      elif i == j:
28          for k in range(i):
29              res -= matrix_l(i, k) * matrix_r(k, i)
30      else:
31          for k in range(j):
32              res -= matrix_l(i, k) * matrix_r(k, j)

```

```

33     return round(res, 6)
34
35
36 def matrix_r(i, j):
37     if i > j:
38         return 0
39     if i == j:
40         return 1
41     if j == 0:
42         return A[i][j] / matrix_l(0, 0)
43     else:
44         res = A[i][j]
45         for t in range(i):
46             res -= matrix_l(i, t) * matrix_r(t, j)
47         res /= matrix_l(i, i)
48     return round(res, 6)
49
50
51 n = len(B)
52 Z = np.zeros(n)
53 Z[0] = B[0] / matrix_l(0, 0)
54 for i in range(n):
55     Z[i] = (B[i] - sum(matrix_l(i, j) * Z[j] for j in
56 range(i))) / matrix_l(i, i)
57
56
57 X = Z.copy()
58 for i in range(len(X) - 2, -1, -1):
59     for k in range(i + 1, len(X)):
60         X[i] -= matrix_r(i, k) * X[k]
61     X[i] = round(X[i], 5)
62
63 print('Ответ. Матрица X:')
64 print(X)

```

Результат:

Матрица A:

5	7	6	5	23
7	10	8	7	32
6	8	10	9	33
5	7	9	10	31

Ответ. Матрица X:
[1. 1. 1. 1.]

Ссылка на код: [hMps://replit.com/@nasirdn/TKM-m'-zakablukovaAE-kostylievaEP](https://replit.com/@nasirdn/TKM-m'-zakablukovaAE-kostylievaEP)

Скринкаст: <https://disk.yandex.ru/i/mBk7c512kkJd6w>

Вывод: в ходе работы была использован метод треугольной факторизации для решения системы линейных уравнений. В результате работы программы после ввода матрицы мы получаем множество решений заданной матрицы.

Лабораторная работа №3

Решение систем линейных уравнений методом треугольной факторизации

Выполнила Закаблуква А.Э.

Постановка задачи:

Разработать программу для решения систем линейных уравнений методом треугольной факторизации.

Математическая модель:

1. Нахождение корне СЛУ последовательным решением 2-х систем с полученными треугольными матрицами L и R

$$LZ = B \quad \text{и} \quad RX = Z$$

$$z_1 = \frac{b_1}{l_{11}}; \quad z_i = \left(b_i - \sum_{k=1}^{i-1} l_{ik} z_k \right) / l_{ii} \quad i = 2 \div n$$

$$x_n = z_n; \quad x_i = z_i - \sum_{j=i+1}^n r_{ij} x_j \quad i = (n-1) \div 1$$

2. Перемножение матриц

$$r_{ii} = 1$$

$$l_{ii} = a_{ii} - \sum_{j=1}^{i-1} l_{ij} r_{ji} \quad i =$$

$$l_{ti} = a_{ti} - \sum_{j=1}^{i-1} l_{tj} r_{ji}$$

$$r_{it} = \left(a_{it} - \sum_{j=1}^{i-1} l_{ij} r_{jt} \right) / l_{ii}$$

Код программы:

```

1  #Метод треугольной факторизации
2
3  import numpy as np
4  from tabulate import tabulate
5
6  #Ввод данных
7  n = 4
8
9  AB = np.array([[5, 7, 6, 5, 23], [7, 10, 8, 7, 32], [6, 8,
10     10, 9, 33],
11     [5, 7, 9, 10, 31]])
12
13  print("Матрица A: ")
14  print(tabulate(AB, tablefmt="fancy_grid"))
15
16  #Матрица A
17  A = np.array([[5, 7, 6, 5], [7, 10, 8, 7], [6, 8, 10, 9], [5,
18     7, 9, 10]])
19
20  B = np.array([23, 32, 33, 31])
21
22  def matrix_l(i, j):
23      if j > i:
24          return 0
25      res = A[i][j]
26      if j == 0:
27          return res
28      elif i == j:
29          for k in range(i):
30              res -= matrix_l(i, k) * matrix_r(k, i)
31      else:
32          for k in range(j):
33              res -= matrix_l(i, k) * matrix_r(k, j)

```

```

33     return round(res, 6)
34
35
36 def matrix_r(i, j):
37     if i > j:
38         return 0
39     if i == j:
40         return 1
41     if j == 0:
42         return A[i][j] / matrix_l(0, 0)
43     else:
44         res = A[i][j]
45         for t in range(i):
46             res -= matrix_l(i, t) * matrix_r(t, j)
47         res /= matrix_l(i, i)
48     return round(res, 6)
49
50
51 n = len(B)
52 Z = np.zeros(n)
53 Z[0] = B[0] / matrix_l(0, 0)
54 for i in range(n):
55     Z[i] = (B[i] - sum(matrix_l(i, j) * Z[j] for j in
56 range(i))) / matrix_l(i, i)
57
56
57 X = Z.copy()
58 for i in range(len(X) - 2, -1, -1):
59     for k in range(i + 1, len(X)):
60         X[i] -= matrix_r(i, k) * X[k]
61     X[i] = round(X[i], 5)
62
63 print('Ответ. Матрица X:')
64 print(X)

```

Результат:

Матрица A:

5	7	6	5	23
7	10	8	7	32
6	8	10	9	33
5	7	9	10	31

Ответ. Матрица X:
[1. 1. 1. 1.]

Ссылка на код: [hMps://replit.com/@nasirdn/TKM-m'-zakablukovaAE-kostylievaEP](https://replit.com/@nasirdn/TKM-m'-zakablukovaAE-kostylievaEP)

Скринкаст: <https://disk.yandex.ru/i/mBk7c512kkJd6w>

Вывод: в ходе работы была использован метод треугольной факторизации для решения системы линейных уравнений. В результате работы программы после ввода матрицы мы получаем множество решений заданной матрицы.

Лабораторная работа №4.

Решение систем линейных уравнений методом квадратного корня

Выполнила Костылева Э.П.

Постановка задачи:

Разработать программу для решения систем линейных уравнений методом квадратного корня.

Математическая модель:

$$AX=B$$

$$A = L \cdot R = R^T \cdot R$$

Нахождение корней СЛУ $AX=B$

$$R^T Z = B$$

$$R X = Z$$

Нахождение элементов матрицы Z

$$Z_1 = b_1 / \gamma_{11}$$

$$Z_i = (b_i - \sum_{k=1}^{i-1} \gamma_{ki} Z_k) / \gamma_{ii}$$

Нахождение элементов матрицы x

$$X_i = (Z_i - \sum_j^j L_T * X_j) / L_T[i][i]$$

Код программы:

```

1  #Метод квадратного корня
2
3  import numpy as np
4  from tabulate import tabulate
5
6  #Матрица A
7  A = np.array([[5, 7, 6, 5, 23], [7, 10, 8, 7, 32], [6, 8, 10,
8  9, 33],
9  [5, 7, 9, 10, 31]])
10
11 print("Матрица A: ")
12 print(tabulate(A, tablefmt="fancy_grid"))
13
14 def matrix_l(A):
15     n = len(A)
16     L = np.zeros((n, n))
17
18     for i in range(n):
19         for j in range(i + 1):
20             if i == j:
21                 sum_term = sum(L[i][k]**2 for k in range(j))
22                 L[i][j] = np.sqrt(A[i][i] - sum_term)
23             else:
24                 sum_term = sum(L[i][k] * L[j][k] for k in range(j))
25                 L[i][j] = (A[i][j] - sum_term) / L[j][j]
26     return L
27
28
29 def matrix_z(L, B):
30     n = len(B)
31     Z = np.zeros(n)
32

```

```

33     for i in range(n):
34         Z[i] = (B[i] - sum(L[i][j] * Z[j] for j in range(i))) /
L[i][i]
35     return Z
36
37
38 def matrix_x(L_T, Z):
39     n = len(Z)
40     X = np.zeros(n)
41
42     for i in range(n - 1, -1, -1):
43         X[i] = (Z[i] - sum(L_T[i][j] * X[j] for j in range(i + 1,
n))) / L_T[i][i]
44     return X
45
46
47 print("Матрица L: ")
48 L = matrix_l(A[:, :-1])
49 print(tabulate(L, tablefmt="fancy_grid"))
50
51 print("Матрица L^T: ")
52 L_T = L.transpose()
53 print(tabulate(L_T, tablefmt="fancy_grid"))
54
55 print("Матрица Z: ")
56 B = A[:, -1]
57 Z = matrix_z(L, B)
58 print(Z)
59
60 print('Ответ. Матрица X:')
61 X = matrix_x(L_T, Z)
62 print(X)

```

Результат:

Матрица A:

5	7	6	5	23
7	10	8	7	32
6	8	10	9	33
5	7	9	10	31

Матрица L:

2.23607	0	0	0
3.1305	0.447214	0	0
2.68328	-0.894427	1.41421	0
2.23607	0	2.12132	0.707107

Матрица L^T :

2.23607	3.1305	2.68328	2.23607
0	0.447214	-0.894427	0
0	0	1.41421	2.12132
0	0	0	0.707107

Матрица Z:

[10.2859127 -0.4472136 3.53553391 0.70710678]

Ответ. Матрица X:

[1. 1. 1. 1.]

Ссылка на код: [hMps://replit.com/@nasirdn/TKM-mkkzakablukovaAEkostylievaEP#main.py](https://replit.com/@nasirdn/TKM-mkkzakablukovaAEkostylievaEP#main.py)

Скринкаст: <https://disk.yandex.ru/i/mBk7c512kkJd6w>

Вывод: в ходе работы был использован метод квадратного корня для решения системы линейных уравнений. В результате работы программы после ввода матрицы мы получаем матрицу L, ее транспонированную версию, матрицу Z и множество решений первоначальной матрицы.

Лабораторная работа №4.

Решение систем линейных уравнений методом квадратного корня

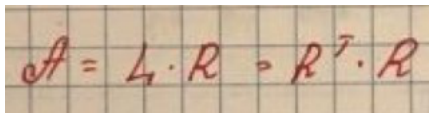
Выполнила Закаблукова А.Э.

Постановка задачи:

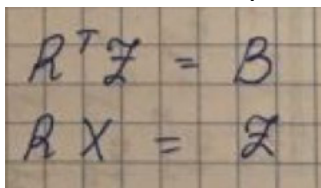
Разработать программу для решения систем линейных уравнений методом квадратного корня.

Математическая модель:

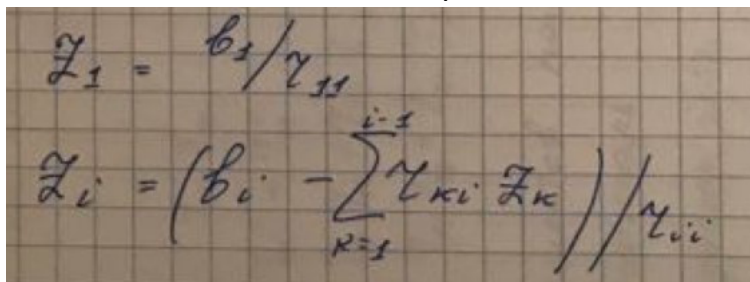
$$AX=B$$


$$A = L \cdot R = R^T \cdot R$$

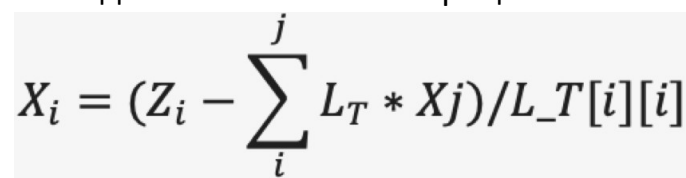
Нахождение корней СЛУ $AX=B$


$$\begin{aligned} R^T Z &= B \\ R X &= Z \end{aligned}$$

Нахождение элементов матрицы Z


$$\begin{aligned} Z_1 &= b_1 / r_{11} \\ Z_i &= (b_i - \sum_{k=1}^{i-1} r_{ki} Z_k) / r_{ii} \end{aligned}$$

Нахождение элементов матрицы x


$$X_i = (Z_i - \sum_j^j L_T * X_j) / L_T[i][i]$$

Код программы:

```

1  #Метод квадратного корня
2
3  import numpy as np
4  from tabulate import tabulate
5
6  #Матрица A
7  A = np.array([[5, 7, 6, 5, 23], [7, 10, 8, 7, 32], [6, 8, 10,
8  9, 33],
9  [5, 7, 9, 10, 31]])
10
11 print("Матрица A: ")
12 print(tabulate(A, tablefmt="fancy_grid"))
13
14 def matrix_l(A):
15     n = len(A)
16     L = np.zeros((n, n))
17
18     for i in range(n):
19         for j in range(i + 1):
20             if i == j:
21                 sum_term = sum(L[i][k]**2 for k in range(j))
22                 L[i][j] = np.sqrt(A[i][i] - sum_term)
23             else:
24                 sum_term = sum(L[i][k] * L[j][k] for k in range(j))
25                 L[i][j] = (A[i][j] - sum_term) / L[j][j]
26     return L
27
28
29 def matrix_z(L, B):
30     n = len(B)
31     Z = np.zeros(n)
32

```

```

33     for i in range(n):
34         Z[i] = (B[i] - sum(L[i][j] * Z[j] for j in range(i))) /
L[i][i]
35     return Z
36
37
38 def matrix_x(L_T, Z):
39     n = len(Z)
40     X = np.zeros(n)
41
42     for i in range(n - 1, -1, -1):
43         X[i] = (Z[i] - sum(L_T[i][j] * X[j] for j in range(i + 1,
n))) / L_T[i][i]
44     return X
45
46
47 print("Матрица L: ")
48 L = matrix_l(A[:, :-1])
49 print(tabulate(L, tablefmt="fancy_grid"))
50
51 print("Матрица L^T: ")
52 L_T = L.transpose()
53 print(tabulate(L_T, tablefmt="fancy_grid"))
54
55 print("Матрица Z: ")
56 B = A[:, -1]
57 Z = matrix_z(L, B)
58 print(Z)
59
60 print('Ответ. Матрица X:')
61 X = matrix_x(L_T, Z)
62 print(X)

```

Результат:

Матрица A:

5	7	6	5	23
7	10	8	7	32
6	8	10	9	33
5	7	9	10	31

Матрица L:

2.23607	0	0	0
3.1305	0.447214	0	0
2.68328	-0.894427	1.41421	0
2.23607	0	2.12132	0.707107

Матрица L^T :

2.23607	3.1305	2.68328	2.23607
0	0.447214	-0.894427	0
0	0	1.41421	2.12132
0	0	0	0.707107

Матрица Z:

[10.2859127 -0.4472136 3.53553391 0.70710678]

Ответ. Матрица X:

[1. 1. 1. 1.]

Ссылка на код: [hMps://replit.com/@nasirdn/TKM-mkkzakablukovaAEkostylievaEP#main.py](https://replit.com/@nasirdn/TKM-mkkzakablukovaAEkostylievaEP#main.py)

Скринкаст: <https://disk.yandex.ru/i/mBk7c512kkJd6w>

Вывод: в ходе работы был использован метод квадратного корня для решения системы линейных уравнений. В результате работы программы после ввода матрицы мы получаем матрицу L, ее транспонированную версию, матрицу Z и множество решений первоначальной матрицы.

